Local Language Phrase Practice Tool

Backend Specification: overall design + 4 member briefs

Java 21 | Plain JDBC | MySQL | Console test harness (frontend plugs in later)

Skill Lab - Advanced Programming using OOP with Java | Project demo and viva: 28 Oct 2026

How to use this document (for the team and for an AI assistant)

- **Step 1:** paste the project problem statement first, then paste this whole document. Say: "Generate the code for the whole backend" or "Generate the code for Member N only".
- **Step 2:** ask for **one member at a time**, and paste each member's output into the shared repo. The contract in Section 5 and the PhraseApp listing in Member 4's brief is frozen, so the four parts fit together without edits.
- **Step 3:** run the checklist in Section 11 on the combined project. Section 12 holds the viva questions.
- Ready-made prompts are in Section 13. A seed SQL file (`phrase_db_setup.sql`) comes with this PDF because Hindi (Devanagari) script cannot be shown reliably inside a PDF.

Rules for whoever writes the code (human or AI)

- Output **complete files** with `package` and `import` lines. No "..." placeholders and no "rest of the code" comments.
- Keep every **class name, method name, parameter type, return type and thrown exception exactly as written** in this document.
- Use only the concepts in Section 2 (the Skill Lab syllabus). If two designs are possible, choose the simpler one.
- Put a one-line comment above every class and every method saying what it does, in plain English, so it can be explained in the viva.
- Never build SQL by joining strings with user input. Always use `PreparedStatement`.
- Read and write all files in **UTF-8** because the data contains Hindi (Devanagari) script.

1. Problem statement and what it needs

Local Language Phrase Practice Tool. Create an offline phrase-learning tool for a selected local language. Each entry includes a phrase, translation, optional transliteration, and category. Users practice through randomized quizzes and track their results using a verified sample dataset.

Expected outcome: search phrases by situation; conduct translation quizzes; record results; summarize progress by category; allow additions and corrections to verified entries.

Requirement	How the design meets it	Owner
Offline tool, selected local language	Local MySQL database, no internet. Language is **Hindi** (data lives in the database, so another language is just different rows).	M1
Entry = phrase, translation, optional transliteration, category	`Phrase` class and `phrases` table; transliteration may be empty.	M2 / M1
Verified sample dataset	35 seeded phrases in 5 categories with `verified = TRUE`. Users' new entries start as `verified = FALSE`.	M1
Search phrases by situation	`searchByCategory` (situation = category) and `searchByKeyword`.	M2
Randomized translation quizzes	`Collections.shuffle`; typing mode and multiple-choice mode.	M3
Record results	Every finished quiz is saved to `quiz_results`.	M4
Summarize progress by category	Attempts, correct/total and accuracy % per category, plus the weakest category.	M4
Additions and corrections	`addPhrase`, `correctPhrase`, `setVerified` with validation and duplicate checks.	M2

2. Technical rules (what the code may and may not use)

Item	Decision
Language / tools	Java 21, IntelliJ or Eclipse or VS Code, MySQL 8, `mysql-connector-j` jar in `lib/`.
Allowed concepts	Classes, constructors, encapsulation, static members, inheritance, abstract classes, interfaces, polymorphism, packages, custom exceptions, try-catch, `ArrayList`, `HashMap`, `TreeMap`, `TreeSet`, generics, `Collections.shuffle`, `Random`, file handling (`BufferedReader`, `BufferedWriter`), simple lambdas and `stream().filter().collect()`, JDBC (`DriverManager`, `PreparedStatement`, `ResultSet`).
Not allowed	Spring, Hibernate, Lombok, threads, reflection, complicated streams (`groupingBy`, `flatMap`), design patterns beyond DAO + a simple facade.
Errors	Business rule problems use 3 **checked** exceptions (`InvalidInputException`, `DuplicatePhraseException`, `PhraseNotFoundException`). SQL problems are wrapped into one **unchecked** `DatabaseException`, so the services and the future UI never see `java.sql`.
Frontend	Not built yet. The future UI (Swing or web) will call **only** the `PhraseApp` class. A console `Main` is used for testing now.
Dates	`java.time.LocalDate` in Java, `DATE` in MySQL.

3. Architecture and file ownership

```
PhraseTool/
 |- lib/            mysql-connector-j-8.x.jar
 |- sql/            phrase_db_setup.sql          (schema + 35 verified Hindi phrases)
 |- data/           exported CSV files go here
 '- src/
     |- model/      Phrase, QuizResult, CategoryStat
     |- exception/  InvalidInputException, DuplicatePhraseException,
     |                PhraseNotFoundException, DatabaseException
     |- dao/        DBConnection, PhraseDao, PhraseDaoImpl, InMemoryPhraseDao,
     |                ResultDao, ResultDaoImpl, InMemoryResultDao
     |- service/    PhraseService, Question, TypingQuestion, ChoiceQuestion,
     |                QuizService, ProgressService
     |- util/       CsvUtil
     '- app/        PhraseApp (facade), Main (console test menu)

Call flow:   Main / future UI  ->  PhraseApp  ->  Services  ->  DAO interfaces  ->  MySQL
```

Member	Focus	Files owned
M1	Database and DAO (JDBC)	`sql/phrase_db_setup.sql`, `DBConnection`, `PhraseDao`, `PhraseDaoImpl`, `InMemoryPhraseDao`
M2	Phrase model, validation, search	`Phrase`, 4 exception classes, `PhraseService`
M3	Quiz engine (OOP showcase)	`Question`, `TypingQuestion`, `ChoiceQuestion`, `QuizService`
M4	Progress, files, facade, integration	`QuizResult`, `CategoryStat`, `ResultDao`, `ResultDaoImpl`, `InMemoryResultDao`, `ProgressService`, `CsvUtil`, `PhraseApp`, `Main`

Layering rule: a class may call only the layer directly below it. `Main` calls `PhraseApp`; `PhraseApp` calls services; services call DAO interfaces. Services never write SQL and the DAO never validates input.

4. Database schema and verified sample dataset

```
CREATE DATABASE IF NOT EXISTS phrase_db CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
USE phrase_db;

CREATE TABLE IF NOT EXISTS phrases (
    id              INT AUTO_INCREMENT PRIMARY KEY,
    phrase          VARCHAR(200) NOT NULL,
    translation     VARCHAR(200) NOT NULL,
    transliteration VARCHAR(200) NOT NULL DEFAULT '',
    category        VARCHAR(50)  NOT NULL,
    verified        BOOLEAN      NOT NULL DEFAULT FALSE,
    UNIQUE KEY uq_phrase (phrase)
);

CREATE TABLE IF NOT EXISTS quiz_results (
    id        INT AUTO_INCREMENT PRIMARY KEY,
    quiz_date DATE        NOT NULL,
    category  VARCHAR(50) NOT NULL,      -- a category name, or 'ALL' for a mixed quiz
    correct   INT         NOT NULL,
    total     INT         NOT NULL
);
```

- Categories (the "situations"): **Greetings, Travel, Food, Shopping, Emergency**, 7 phrases each, all `verified = TRUE` in the seed.
- The Hindi (Devanagari) script for each row is in the attached `phrase_db_setup.sql`. The table below shows the transliteration and meaning so the data can be checked.
- **Before the demo, ask a Hindi speaker (or a dictionary) to check the script and transliteration.** That check is what makes the dataset "verified".

#	Category	Transliteration	English meaning
1	Greetings	namaste	Hello
2	Greetings	aap kaise hain?	How are you?
3	Greetings	main theek hoon	I am fine
4	Greetings	dhanyavaad	Thank you
5	Greetings	suprabhaat	Good morning
6	Greetings	shubh raatri	Good night
7	Greetings	aapka naam kya hai?	What is your name?
8	Travel	bus stop kahaan hai?	Where is the bus stop?
9	Travel	railway station kahaan hai?	Where is the railway station?
10	Travel	seedhe jaaiye	Go straight
11	Travel	daayen mudiye	Turn right
12	Travel	baayen mudiye	Turn left
13	Travel	kripya rukiye	Please stop
14	Travel	yah kahaan hai?	Where is this?
15	Food	mujhe paani chaahiye	I need water
16	Food	mujhe khaana chaahiye	I want food
17	Food	kripya chaay dijiye	Please give tea
18	Food	yah bahut swaadisht hai	It is very tasty
19	Food	thoda namak chaahiye	I need a little salt
20	Food	bill dijiye	Please give the bill
21	Food	mujhe bhookh lagi hai	I am hungry
22	Shopping	yah kitne ka hai?	How much is this?
23	Shopping	thoda kam kijiye	Please reduce the price a little
24	Shopping	mujhe yah nahin chaahiye	I do not want this
25	Shopping	doosra dikhaaiye	Show me another one

#	Category	Transliteration	English meaning
26	Shopping	thaila dijiye	Please give a bag
27	Shopping	mujhe yah pasand aaya	I liked this
28	Shopping	yah achchha hai	This is good
29	Emergency	madad kijiye!	Help me!
30	Emergency	police ko bulaaiye	Call the police
31	Emergency	doctor ko bulaaiye	Call a doctor
32	Emergency	meri tabiyat theek nahin hai	I am not well
33	Emergency	main raasta bhool gaya hoon	I am lost
34	Emergency	aspataal kahaan hai?	Where is the hospital?
35	Emergency	aag!	Fire!

5. Shared contract (frozen today: do not change without telling everyone)

Everyone copies these classes and interfaces as they are. Getters and setters follow normal Java naming (`getPhrase()`, `setPhrase(...)`, `isVerified()`).

5.1 Model classes (package `model`)

```
package model;
public class Phrase {                       // owner: M2
    private int id;                         // 0 = not saved yet
    private String phrase;                  // text in the local language (Devanagari script)
    private String translation;             // English meaning
    private String transliteration;         // Latin spelling, may be "" (optional)
    private String category;                // e.g. "Travel"
    private boolean verified;

    public Phrase() { }
    public Phrase(String phrase, String translation, String transliteration, String category) {
        this(0, phrase, translation, transliteration, category, false);   // new entry
    }
    public Phrase(int id, String phrase, String translation, String transliteration,
                  String category, boolean verified) { /* assign all fields with this. */ }
    // getters + setters for all 6 fields (boolean getter is isVerified)
    // toString(): "[id] phrase (transliteration) = translation | category | VERIFIED/UNVERIFIED"
}
```

```
package model;
import java.time.LocalDate;
public class QuizResult {                   // owner: M4
    private int id; private LocalDate quizDate; private String category;
    private int correct; private int total;
    public QuizResult(String category, int correct, int total) { /* id=0, date=LocalDate.now() */ }
    public QuizResult(int id, LocalDate quizDate, String category, int correct, int total) { }
    // getters for all fields
    public double getPercentage() { /* total == 0 ? 0 : correct * 100.0 / total */ }
}
```

```
package model;
public class CategoryStat {                 // owner: M4  (one row of the progress summary)
    private String category; private int quizzes; private int correct; private int total;
    public CategoryStat(String category, int quizzes, int correct, int total) { }
    // getters for all fields
    public double getAccuracy() { /* total == 0 ? 0 : correct * 100.0 / total */ }
}
```

5.2 Exceptions (package `exception`)

```
public class InvalidInputException   extends Exception { public InvalidInputException(String msg)   { super(msg); } }
public class DuplicatePhraseException extends Exception { public DuplicatePhraseException(String msg){ super(msg); } }
public class PhraseNotFoundException  extends Exception { public PhraseNotFoundException(String msg) { super(msg); } }
public class DatabaseException extends RuntimeException {      // wraps SQLException
    public DatabaseException(String msg, Throwable cause) { super(msg, cause); }
}
```

5.3 DAO interfaces (package `dao`)

```
public interface PhraseDao {
    void insert(Phrase p);                      // saves p; sets the generated id on p
    boolean update(Phrase p);                   // updates by p.getId(); true if a row changed
    Phrase findById(int id);                    // null if not found
    List<Phrase> findAll();                     // ordered by category, then id
    List<Phrase> findByCategory(String category);// category compared ignoring case
}

public interface ResultDao {
    void save(QuizResult r);                    // inserts one row
    List<QuizResult> findAll();                 // oldest first
}
```

5.4 Service method signatures

```
public class PhraseService {                                                    // M2
    public PhraseService(PhraseDao dao)
    public void addPhrase(Phrase p)           throws InvalidInputException, DuplicatePhraseException
    public void correctPhrase(Phrase p)       throws InvalidInputException, PhraseNotFoundException,
                                                     DuplicatePhraseException
    public void setVerified(int id, boolean v) throws PhraseNotFoundException
    public Phrase getById(int id)             throws PhraseNotFoundException
    public List<Phrase> getAllPhrases()
    public List<Phrase> getVerifiedPhrases()
    public List<String> getCategories()                    // unique, sorted A-Z
    public List<Phrase> searchByCategory(String category)  // blank or "ALL" returns everything
    public List<Phrase> searchByKeyword(String keyword)    throws InvalidInputException
}
public abstract class Question {                                                // M3
    public Question(Phrase phrase)
    public String getText()                   // "Translate: <phrase> (<transliteration>)"
    public String getCategory()
    public String getCorrectAnswer()          // the translation
    public abstract boolean checkAnswer(String answer)
}
public class TypingQuestion extends Question { ... }
public class ChoiceQuestion extends Question {
    public ChoiceQuestion(Phrase phrase, List<String> options)
    public List<String> getOptions()
}
public class QuizService {
    public QuizService(PhraseService phraseService)
    public List<Question> createQuiz(String category, int count, String mode) throws InvalidInputException
    public QuizResult evaluate(List<Question> questions, List<String> answers) throws InvalidInputException
}
public class ProgressService {                                                  // M4
    public ProgressService(ResultDao dao)
    public void saveResult(QuizResult r)
    public List<CategoryStat> getCategorySummary()
    public CategoryStat getWeakestCategory()   // null if no results
}
public class CsvUtil {
    public static void exportPhrases(List<Phrase> phrases, String filePath) throws IOException
    public static List<Phrase> importPhrases(String filePath) throws IOException, InvalidInputException
}
```

6. Business rules (the behaviour every module must follow)

#	Rule
R1	**Validation:** `phrase`, `translation` and `category` must be non-null and non-empty after `trim()`. `transliteration` may be empty (store `""`, never null). Max length: 200 for phrase, translation and transliteration; 50 for category. Violations throw `InvalidInputException` with a clear message.
R2	**Duplicates:** two entries are duplicates if their `phrase` text is equal ignoring case and surrounding spaces. Adding a duplicate throws `DuplicatePhraseException`. When correcting, a clash with a **different** id throws it too; matching itself is fine.
R3	**New entries are unverified.** `addPhrase` always sets `verified = false`, whatever the caller passed.
R4	**Corrections:** `correctPhrase` replaces phrase, translation, transliteration and category of the entry with that id and **keeps its current verified flag**. Unknown id throws `PhraseNotFoundException`.
R5	**Verification:** `setVerified(id, true/false)` flips only the flag. Only verified phrases are used in quizzes.
R6	**Search:** matching ignores case. Keyword search looks in phrase, translation and transliteration (`contains`). A null or blank keyword throws `InvalidInputException`. Category search with null, blank or "ALL" returns every phrase.
R7	**Quiz setup:** mode is `"TYPING"` or `"CHOICE"` (ignore case); count must be at least 1; category may be a name or `"ALL"`. Pool = verified phrases in that category. Empty pool throws `InvalidInputException`. If count is bigger than the pool, use the pool size.
R8	**Quiz direction:** show the Hindi phrase (plus transliteration if present); the answer is the English `translation`.
R9	**Answer checking:** both sides are normalised (trim, lower-case, collapse spaces, drop trailing `. ? !`) then compared with `equals`. So "thank you." equals "Thank you".
R10	**Multiple choice:** up to 4 options = the correct translation + up to 3 different wrong translations taken from other verified phrases (never equal to the correct one, no repeats), shuffled. At least 2 verified phrases must exist, otherwise `InvalidInputException`.
R11	**Scoring:** `evaluate` needs `questions.size() == answers.size()`. Result category = the questions' category if all are the same, otherwise `"ALL"`.
R12	**Progress:** every submitted quiz is saved. Summary groups saved results by category: quizzes (count), correct (sum), total (sum), accuracy = correct / total. The **weakest category** is the lowest accuracy, ignoring the label `"ALL"` and rows with total 0.
R13	**CSV:** header `id,phrase,translation,transliteration,category,verified`, UTF-8, fields in double quotes. Imported rows are added as **unverified** and duplicates are skipped (counted, not fatal).

MEMBER 1: Database and DAO

Unit III: JDBC architecture, PreparedStatement, ResultSet, CRUD. Concepts: interface vs implementation, try-with-resources.

Files to deliver

- `sql/phrase_db_setup.sql` (schema from Section 4 + 35 `INSERT` rows, all `verified = TRUE`; the attached file is a ready start).
- `dao/DBConnection.java`, `dao/PhraseDao.java`, `dao/PhraseDaoImpl.java`, `dao/InMemoryPhraseDao.java`.

Deadline plan

- **Oct 25 night / Oct 26 morning:** `PhraseDao` + `InMemoryPhraseDao` pushed first, because M2, M3 and M4 test against it.
- **Oct 26 evening:** schema loaded in MySQL, `PhraseDaoImpl` working, `DaoTest` main passes.
- **Oct 27 morning (JDBC lab):** help the team switch from `InMemoryPhraseDao` to `PhraseDaoImpl`.

DBConnection

```
package dao;
public class DBConnection {
    private static final String URL  = "jdbc:mysql://localhost:3306/phrase_db"
            + "?useUnicode=true&characterEncoding=UTF-8&useSSL=false&allowPublicKeyRetrieval=true";
    private static final String USER = "root";
    private static final String PASSWORD = "your_password";      // change locally
    public static Connection getConnection() throws SQLException {
        return DriverManager.getConnection(URL, USER, PASSWORD);
    }
}
```

PhraseDaoImpl: exact behaviour

- Implements `PhraseDao`. Every method opens a connection with try-with-resources (`Connection`, `PreparedStatement`, `ResultSet` closed automatically).
- **insert:** `INSERT INTO phrases(phrase, translation, transliteration, category, verified) VALUES (?,?,?,?,?)` using `Statement.RETURN_GENERATED_KEYS`; copy the key into `p.setId(...)`.
- **update:** `UPDATE phrases SET phrase=?, translation=?, transliteration=?, category=?, verified=? WHERE id=?`; return `executeUpdate() > 0`.
- **findById:** `SELECT ... WHERE id=?`; return `null` if no row.
- **findAll:** `SELECT ... ORDER BY category, id`. **findByCategory:** `WHERE LOWER(category) = LOWER(?)`.
- A private helper `mapRow(ResultSet rs)` builds a `Phrase` with the 6-argument constructor; null transliteration becomes `""`.
- Catch `SQLException` and rethrow `new DatabaseException("Could not <action>", e)`. Never print stack traces here.

InMemoryPhraseDao: exact behaviour

- Implements the same `PhraseDao` using an `ArrayList<Phrase>` and an `int nextId = 1`.
- Constructor pre-loads 6 sample phrases (3 verified, 3 unverified, at least 2 categories) so others can test immediately.
- `insert` assigns `nextId++`; `findAll` returns **a new ArrayList** (so callers cannot corrupt the list); `update` replaces the element with the same id.

Test class to include: `dao/DaoTest.java` (has a `main`)

- insert a phrase and print its id; findById; update its translation and read it again;
- findAll count is at least 36 after the seed; findByCategory("travel") returns 7 or more rows (case ignored).

Viva answers

- **Why a DAO interface?** Services depend on `PhraseDao`, not on MySQL, so `InMemoryPhraseDao` and `PhraseDaoImpl` can be swapped without changing any service.
- **Why PreparedStatement?** It pre-compiles the SQL and treats input as data, which prevents SQL injection and handles quotes in phrases.
- **Why wrap SQLException?** The upper layers only know our own `DatabaseException`, so they do not depend on JDBC.
- **Why utf8mb4 and `useUnicode`?** Hindi characters need Unicode in both the table and the connection.

MEMBER 2: Phrase model, validation and search

Unit I and II: encapsulation, constructors, custom exceptions, collections, basic lambda.

Files to deliver

- `model/Phrase.java` (Section 5.1), the 4 classes in `exception/` (Section 5.2), `service/PhraseService.java`.
- **Push** `Phrase` **and the exceptions first (within an hour)**: every other member compiles against them.

PhraseService: exact behaviour

- Field `private final PhraseDao dao;` set by the constructor.
- **addPhrase(p):** `validate(p)` (rule R1) -> trim all text fields in place -> if `isDuplicate(p.getPhrase(), 0)` throw `DuplicatePhraseException("Phrase already exists: ...")` -> `p.setVerified(false)` -> `dao.insert(p)`.
- **correctPhrase(p):** `validate(p)` -> `Phrase old = dao.findById(p.getId())`; if null throw `PhraseNotFoundException("No phrase with id ...")` -> trim fields -> `isDuplicate(p.getPhrase(), p.getId())` -> `p.setVerified(old.isVerified())` -> `dao.update(p)`.
- **setVerified(id, v):** find by id (throw `PhraseNotFoundException` if null), `setVerified(v)`, `dao.update(...)`.
- **getById(id):** find or throw `PhraseNotFoundException`.
- **getAllPhrases():** `dao.findAll()`. **getVerifiedPhrases():** loop (or `stream().filter(Phrase::isVerified).collect(Collectors.toList())`).
- **getCategories():** put every category into a `TreeSet<String>` (sorted, unique), return `new ArrayList<>(set)`. Because MySQL and Java may differ in case, store the first spelling seen: use a `TreeMap<String,String>` keyed by `category.toLowerCase()`.
- **searchByCategory(cat):** if null/blank/"ALL" return all; otherwise `dao.findByCategory(cat.trim())`.
- **searchByKeyword(kw):** blank -> `InvalidInputException("Keyword cannot be empty")`; lower-case the keyword; return phrases where lower-cased `phrase`, `translation` or `transliteration` **contains** it.

```
// private helpers (exact logic)
private void validate(Phrase p) throws InvalidInputException {
    if (p == null) throw new InvalidInputException("Phrase is missing");
    if (isBlank(p.getPhrase()))      throw new InvalidInputException("Phrase text is required");
    if (isBlank(p.getTranslation())) throw new InvalidInputException("Translation is required");
    if (isBlank(p.getCategory()))    throw new InvalidInputException("Category is required");
    // + length checks from rule R1 (also: null transliteration -> "")
}
private boolean isDuplicate(String text, int ignoreId) {
    for (Phrase x : dao.findAll())
        if (x.getId() != ignoreId && x.getPhrase().trim().equalsIgnoreCase(text.trim())) return true;
    return false;
}
```

Test class to include: `service/PhraseServiceTest.java`

- uses `new PhraseService(new InMemoryPhraseDao())` and prints PASS/FAIL for: add valid, add duplicate (different case), add empty translation,
- correct unknown id, keyword "WATER" finds "I need water", category "travel" (lower case) works, `getCategories()` has no repeats.

Viva answers

- **Why validation in the service, not the DAO?** Rules are business logic; the DAO only stores. Bad data is stopped before it reaches the database.
- **Why checked custom exceptions?** The caller is forced to handle "duplicate" or "not found" and show a proper message instead of crashing.
- **Why `TreeSet`/`TreeMap`?** They keep categories unique and sorted without extra code.
- **Why does a new phrase start unverified?** The verified dataset is the trusted source; user additions must be reviewed first.

MEMBER 3: Quiz engine

Unit I: inheritance, abstraction, method overriding, polymorphism. Also generics (`List<Question>`), `Collections.shuffle`, `Random`.

Files to deliver

- `service/Question.java` (abstract), `service/TypingQuestion.java`, `service/ChoiceQuestion.java`, `service/QuizService.java`.
- Question classes depend only on `Phrase`, so they can be tested on day 1 with hand-made `Phrase` objects.

Question hierarchy

```
public abstract class Question {
    protected Phrase phrase;
    public Question(Phrase phrase) { this.phrase = phrase; }
    public String getText() {
        String t = "Translate: " + phrase.getPhrase();
        if (phrase.getTransliteration() != null && !phrase.getTransliteration().isEmpty())
            t += " (" + phrase.getTransliteration() + ")";
        return t;
    }
    public String getCategory()      { return phrase.getCategory(); }
    public String getCorrectAnswer() { return phrase.getTranslation(); }
    public abstract boolean checkAnswer(String answer);          // each subclass decides

    protected static String normalize(String s) {                // rule R9
        if (s == null) return "";
        String t = s.trim().toLowerCase().replaceAll("\\s+", " ");
        return t.replaceAll("[.?!]+$", "");
    }
}
```

- **TypingQuestion:** `checkAnswer(a)` returns `normalize(a).equals(normalize(getCorrectAnswer()))`.
- **ChoiceQuestion:** extra field `List<String> options`; constructor `(Phrase, List<String>)`; `getOptions()` returns the list; `checkAnswer(a)` uses the same normalised comparison against the correct translation. (The console or UI converts a chosen number into the option text before calling it.) Override `getText()` is **not** needed.

QuizService.createQuiz(category, count, mode): follow these steps

- 1. Validate `mode` ("TYPING" or "CHOICE", ignore case) and `count >= 1`; otherwise `InvalidInputException`.
- 2. `all = phraseService.getVerifiedPhrases()`. If `category` is null/blank/"ALL" use `all`, else keep only phrases whose category `equalsIgnoreCase` it. Copy into a **new** `ArrayList` called `pool`.
- 3. If `pool` is empty throw `InvalidInputException("No verified phrases found for category ...")`. If mode is CHOICE and `all.size() < 2` throw `InvalidInputException`.
- 4. `Collections.shuffle(pool)`; `n = Math.min(count, pool.size())`; take the first `n`.
- 5. For each chosen phrase: TYPING -> `new TypingQuestion(p)`; CHOICE -> `new ChoiceQuestion(p, buildOptions(p, all))`.
- 6. Return them in a `List<Question>` (a mix of subclasses is fine; that is polymorphism).

```
private List<String> buildOptions(Phrase correct, List<Phrase> all) {
    List<String> wrong = new ArrayList<>();
    for (Phrase x : all) {
        String t = x.getTranslation();
        boolean sameAsCorrect = t.trim().equalsIgnoreCase(correct.getTranslation().trim());
        boolean alreadyIn = false;
        for (String w : wrong) if (w.equalsIgnoreCase(t.trim())) alreadyIn = true;
        if (!sameAsCorrect && !alreadyIn) wrong.add(t);
    }
    Collections.shuffle(wrong);
    List<String> options = new ArrayList<>(wrong.subList(0, Math.min(3, wrong.size())));
    options.add(correct.getTranslation());
    Collections.shuffle(options);
    return options;
}
```

QuizService.evaluate(questions, answers)

- If either list is null or the sizes differ, throw `InvalidInputException`.
- Loop `i`: `if (questions.get(i).checkAnswer(answers.get(i))) correct++;` (the call is polymorphic: it runs the right subclass method).
- Category = first question's category; if any question has a different one use `"ALL"` (rule R11). Return `new QuizResult(category, correct, total)`. **Do not save it here**; `PhraseApp` saves via `ProgressService`.

Test class to include: `service/QuizServiceTest.java`

- with `PhraseService(new InMemoryPhraseDao())`: a TYPING quiz of 3; a CHOICE quiz where each question has the correct answer exactly once and no repeated options;
- count 100 returns only as many as exist; unknown category throws; answering " THANK YOU. " is accepted for "Thank you"; a score of 2 out of 3 is computed correctly.

Viva answers

- **Where is polymorphism?** `List<Question>` holds both types; `checkAnswer` is overridden and the right version runs at run time.
- **Why an abstract class?** `Question` has shared code (text, category, normalize) but "how to check" differs, so it stays abstract.
- **How are quizzes random?** `Collections.shuffle` on a copy of the pool, then take the first `n`; options are shuffled again.
- **Why only verified phrases?** Learners must practise trusted data, not unreviewed entries.

MEMBER 4: Progress, files, facade and integration

Unit II and III: Map, file handling, JDBC for results, facade class, console main, final testing and documentation.

Files to deliver

- `model/QuizResult.java`, `model/CategoryStat.java` (Section 5.1; **push these first**, M3 needs `QuizResult`).
- `dao/ResultDao.java`, `dao/ResultDaoImpl.java` (JDBC), `dao/InMemoryResultDao.java` (an `ArrayList`).
- `service/ProgressService.java`, `util/CsvUtil.java`, `app/PhraseApp.java`, `app/Main.java`, plus the project README and test log.

ResultDaoImpl / InMemoryResultDao

- **save:** `INSERT INTO quiz_results(quiz_date, category, correct, total) VALUES (?,?,?,?)`, with `java.sql.Date.valueOf(r.getQuizDate())`.
- **findAll:** `SELECT * FROM quiz_results ORDER BY quiz_date, id`; build `QuizResult` using `rs.getDate("quiz_date").toLocalDate()`. Wrap `SQLException` in `DatabaseException`.
- `InMemoryResultDao` keeps an `ArrayList<QuizResult>` with the same two methods so progress can be tested before MySQL is ready.

ProgressService

```
public List<CategoryStat> getCategorySummary() {
    Map<String, int[]> map = new TreeMap<>();            // category -> {quizzes, correct, total}
    for (QuizResult r : resultDao.findAll()) {
        int[] a = map.computeIfAbsent(r.getCategory(), k -> new int[3]);
        a[0] += 1;  a[1] += r.getCorrect();  a[2] += r.getTotal();
    }
    List<CategoryStat> list = new ArrayList<>();
    for (Map.Entry<String, int[]> e : map.entrySet())
        list.add(new CategoryStat(e.getKey(), e.getValue()[0], e.getValue()[1], e.getValue()[2]));
    return list;
}
public CategoryStat getWeakestCategory() {               // rule R12
    CategoryStat weakest = null;
    for (CategoryStat s : getCategorySummary()) {
        if (s.getCategory().equalsIgnoreCase("ALL") || s.getTotal() == 0) continue;
        if (weakest == null || s.getAccuracy() < weakest.getAccuracy()) weakest = s;
    }
    return weakest;
}
```

- `saveResult(r)` just calls `resultDao.save(r)`.

CsvUtil

- **exportPhrases:** `BufferedWriter` over `OutputStreamWriter(new FileOutputStream(path), StandardCharsets.UTF_8)`. Write the header, then one line per phrase: every text field wrapped in quotes with inner quotes doubled (`"` becomes `""`), verified as `true`/`false`.
- **importPhrases:** `BufferedReader` with UTF-8. Skip the header and blank lines. For each line call `splitLine`; if there are fewer than 6 parts throw `InvalidInputException("Bad CSV format on line N")`. Build `new Phrase(phrase, translation, transliteration, category)` (id 0, unverified) and return the list.

```
private static List<String> splitLine(String line) {      // handles commas inside quotes
    List<String> parts = new ArrayList<>();
    StringBuilder cur = new StringBuilder();
    boolean inQuotes = false;
    for (int i = 0; i < line.length(); i++) {
        char c = line.charAt(i);
        if (c == '"') {
            if (inQuotes && i + 1 < line.length() && line.charAt(i + 1) == '"') { cur.append('"'); i++; }
            else inQuotes = !inQuotes;
        } else if (c == ',' && !inQuotes) { parts.add(cur.toString()); cur.setLength(0); }
        else cur.append(c);
    }
    parts.add(cur.toString());
    return parts;
}
```

PhraseApp: the facade the future UI will call

```
package app;
public class PhraseApp {
    private final PhraseService phraseService;
    private final QuizService quizService;
    private final ProgressService progressService;

    public PhraseApp() { this(new PhraseDaoImpl(), new ResultDaoImpl()); }       // real MySQL
    public PhraseApp(PhraseDao phraseDao, ResultDao resultDao) {                  // for testing
        phraseService   = new PhraseService(phraseDao);
        quizService     = new QuizService(phraseService);
        progressService = new ProgressService(resultDao);
    }

    public List<Phrase> getAllPhrases()
    public List<String> getCategories()
    public List<Phrase> searchByCategory(String category)
    public List<Phrase> searchByKeyword(String keyword)          throws InvalidInputException
    public void addPhrase(Phrase p)                              throws InvalidInputException, DuplicatePhraseException
    public void correctPhrase(Phrase p)                          throws InvalidInputException, PhraseNotFoundException,
                                                                        DuplicatePhraseException
    public void setVerified(int id, boolean verified)            throws PhraseNotFoundException
    public List<Question> startQuiz(String category, int count, String mode) throws InvalidInputException
    public QuizResult submitQuiz(List<Question> questions, List<String> answers)
                                                                 throws InvalidInputException  // evaluate + SAVE
    public List<CategoryStat> getProgress()
    public CategoryStat getWeakestCategory()
    public int importCsv(String path)  throws IOException, InvalidInputException  // returns number added
    public void exportCsv(String path) throws IOException
}
```

- Each method is **one or two lines** that delegate to a service. `submitQuiz` = `QuizResult r = quizService.evaluate(...); progressService.saveResult(r); return r;`.
- **importCsv:** for each phrase from `CsvUtil.importPhrases(path)` call `phraseService.addPhrase(p)` inside try-catch; `DuplicatePhraseException` and `InvalidInputException` are caught and skipped; return how many were added.
- **exportCsv:** `CsvUtil.exportPhrases(phraseService.getAllPhrases(), path)`.

Main: console test menu

```
===== Local Language Phrase Practice (Hindi) =====
 1. View all phrases            6. Add a phrase
 2. Search by category          7. Correct a phrase
 3. Search by keyword           8. Verify / unverify a phrase
 4. Take a quiz                 9. Import phrases from CSV
 5. View progress              10. Export phrases to CSV
 0. Exit
```

- At start: `System.setOut(new PrintStream(System.out, true, "UTF-8"))` so Hindi prints correctly; one `Scanner` for the whole program; a `while(true)` loop with `switch`.
- Read numbers with a helper `readInt(prompt)` that loops until `Integer.parseInt` succeeds (no crash on text input).
- Wrap each menu action in try-catch: print the message of `InvalidInputException`, `DuplicatePhraseException`, `PhraseNotFoundException`, `DatabaseException` and `IOException`, then return to the menu.
- Phrase print format: `[id] phrase (transliteration) = translation | category | VERIFIED` (use `Phrase.toString()`).
- **Quiz flow:** show categories plus "ALL", ask for category, number of questions and mode (1 typing, 2 multiple choice). Loop the questions: print `Q1/5: ` + `getText()`. For a `ChoiceQuestion` (check with `instanceof`, allowed in the UI only) print numbered options, read a number, and store the chosen option text; otherwise read a line. Call `submitQuiz`, print `Score: 4/5 (80.0%)` and list the wrong ones with their correct answers.
- **Progress screen:** table of Category | Quizzes | Correct/Total | Accuracy %, then `Weakest category: Food (60.0%)`, or `No quizzes taken yet.` when empty.

Viva answers

- **Why a facade?** The UI knows one class, `PhraseApp`; Swing or a web page can be added without touching services or DAOs.
- **Why `Map<String,int[]>`?** It groups results by category in one pass: key = category, array = {quizzes, correct, total}.
- **Why quotes in the CSV?** Phrases can contain commas, and quoting keeps the columns correct.
- **Why can the same `PhraseApp` run with or without MySQL?** The second constructor accepts any `PhraseDao`/`ResultDao`, so tests use the in-memory versions.

7. Build and run commands

```
# one-time: create tables and seed data
mysql -u root -p < sql/phrase_db_setup.sql

# compile (Linux/Mac use ':' in the classpath, Windows uses ';')
javac -encoding UTF-8 -d out -cp lib/mysql-connector-j-8.x.jar $(find src -name "*.java")

# run
java -Dfile.encoding=UTF-8 -cp out:lib/mysql-connector-j-8.x.jar app.Main
```

- IntelliJ/Eclipse: add the connector jar as a library and set VM option `-Dfile.encoding=UTF-8`.
- If Hindi shows as `???`, the terminal font or encoding is the problem, not the database. Try IntelliJ's run window.

8. Timeline (matches the Skill Lab schedule)

When	M1	M2	M3	M4
Sun 25 Oct (tonight)	Create repo and folders. Push `PhraseDao` + `InMemoryPhraseDao`.	Push `Phrase` + 4 exceptions.	Push `Question`, `TypingQuestion`, `ChoiceQuestion`.	Push `QuizResult`, `CategoryStat`, `ResultDao`, `InMemoryResultDao`.
Mon 26 Oct	Schema + seed in MySQL, `PhraseDaoImpl`, `DaoTest`.	`PhraseService` + test.	`QuizService` + test.	`ProgressService`, `CsvUtil`, `PhraseApp`, `Main` on in-memory DAOs.
Tue 27 Oct (JDBC lab)	Switch to `PhraseDaoImpl`; fix SQL bugs.	Fix review comments; help test.	Fix review comments; help test.	`ResultDaoImpl`, integrate, run all Section 11 tests.
Tue night	Code freeze. Everyone runs the demo once from a clean checkout.			
Wed 28 Oct	Demo + viva (each member explains their own module).			

9. Integration order (so problems show up early)

- 1. M2's `Phrase` + `InMemoryPhraseDao` + `PhraseService` compile together.
- 2. Add M3's `QuizService` on top of `PhraseService`.
- 3. Add M4's `ProgressService` with `InMemoryResultDao`, then `PhraseApp` and `Main`.
- 4. Replace both in-memory DAOs with the JDBC ones by changing only `new PhraseApp()`. Nothing else should change.

10. Optional innovation (only after everything works)

- **Weak-category quiz:** `PhraseApp.startWeakCategoryQuiz(count, mode)` takes `getWeakestCategory()` and calls `startQuiz` with it (falls back to "ALL" when there is no data).
- **Phrase of the day:** pick `verified.get(LocalDate.now().getDayOfYear() % verified.size())`, so the same phrase shows all day.

11. Test checklist (run on the combined project)

ID	Action	Expected result
T1	Start with the seeded DB; menu 1	35 phrases listed, Hindi text readable
T2	Search category `travel` (lower case)	All 7 Travel phrases
T3	Keyword `WATER`	Includes "I need water"
T4	Keyword left blank	Message "Keyword cannot be empty"; menu continues
T5	Add a valid new phrase	Saved; shown as UNVERIFIED
T6	Add the same phrase text again (different case)	Duplicate message, nothing saved
T7	Add with empty translation	Invalid-input message
T8	Correct a phrase's translation, restart the app	Corrected text still there; verified flag unchanged
T9	Correct id 9999	"No phrase with id 9999"
T10	Verify the new phrase, then quiz its category	It can now appear in quizzes
T11	Typing quiz, answer `thank YOU.`	Counted correct
T12	Multiple-choice quiz	4 options each, correct answer appears exactly once, no repeats
T13	Ask for 100 questions in a 7-phrase category	Exactly 7 questions
T14	Quiz in a category with no verified phrases	Clear message, no crash
T15	Finish a quiz, open progress	Attempts increased; accuracy matches the score
T16	Progress on an empty results table	"No quizzes taken yet."
T17	Take quizzes with different scores in 2 categories	Weakest category is the lower-accuracy one
T18	Export CSV, then import the same file	Import reports 0 added (all duplicates); no crash
T19	Type letters at a numeric prompt	Asks again, no crash
T20	Stop MySQL, then run an action	Readable database error message

12. Overall viva questions and short answers

Question	Answer
What does your project do?	An offline Hindi phrase trainer: search by situation, take random quizzes, save results and see accuracy per category, and add or correct phrases.
What is the architecture?	Layered: Main/UI -> PhraseApp (facade) -> services (rules) -> DAO interfaces -> MySQL. Each layer has one job.
Which Unit I concepts are used?	Encapsulation (`Phrase`), inheritance/abstraction/polymorphism (`Question` family), interfaces (DAOs), packages (6 of them).
Which Unit II concepts?	Custom exceptions, collections (`List`, `Map`, `TreeSet`), generics, file handling (CSV), a lambda/stream in search.
Which Unit III concepts?	JDBC with `PreparedStatement` and `ResultSet`, CRUD on `phrases`, insert/select on `quiz_results`.
What makes the dataset "verified"?	A boolean flag. The seed is reviewed and marked true; user additions start false and quizzes use only true entries.
How do you keep it offline?	MySQL runs locally; there are no network calls anywhere.
How will the frontend be added?	It will only call `PhraseApp`; no backend code changes.
What if the DB is unavailable?	The DAO throws `DatabaseException`; the menu prints a message and continues.
How did you test?	Unit-style test mains per module plus the 20 integration tests in Section 11.

13. Copy-paste prompts for an AI assistant

A. Whole backend (one go)

```
I am building a Java backend for the project below. First read the problem statement, then the full
specification (the contract in Section 5 and the PhraseApp listing in Member 4's brief is frozen). Generate COMPLETE, co
EVERY file listed in Section 3, one file per code block, each headed by its path (for example
src/service/PhraseService.java). Keep all class names, method signatures, exceptions and behaviours
exactly as specified, follow business rules R1-R13, use only the allowed concepts in Section 2, and add
a one-line comment above every class and method. Then give the compile and run commands and the
test mains. Finally list 5 viva questions per member.

[PASTE PROBLEM STATEMENT]
[PASTE THIS SPECIFICATION]
```

B. One member at a time (recommended)

```
Using the problem statement and specification below, generate ONLY the files owned by MEMBER <N>
(listed in Section 3 and in that member's brief). Assume every other member's classes exist exactly
as in Section 5. Output complete files with package and imports, no placeholders. Follow the member's
"exact behaviour" bullets and the business rules they refer to. Include the test class the brief asks
for. After the code, explain each class in 2 to 3 simple sentences as I would in a viva, and list
what I must create in MySQL or in the project folders to run it.

[PASTE PROBLEM STATEMENT]
[PASTE THIS SPECIFICATION]
```

C. Review a teammate's code

```
Here is a Java file written for the specification above. Check it against the exact signatures, the
business rules and the viva-explainability limit (no concepts outside Section 2). List every mismatch
as: file, line, problem, fix. Then give the corrected full file.

[PASTE SPEC] [PASTE FILE]
```

14. Final handover checklist

- All four modules compile together with no signature changes.
- `phrase_db_setup.sql` runs on a clean MySQL and the Hindi text is readable.
- Every member can explain their module using the viva answers in their brief.
- A Hindi speaker (or dictionary) has confirmed the 35 seed entries.
- The code is frozen on Tuesday night and the demo has been run once from a clean checkout.